

Integrasjonsguide

DBS

Dato: 11.11.2020
Versjon: 1.15

ENDRINGSHISTORIKK

Versjonsnr	Endre kapittel	Beskrivelse av endring	Initialer
1.0	Alle	Første utkast	
1.1	Alle	Lagt til avsnitt om time out, duplikatkontroll og feilhåndtering Lagt til konverteringstabell meldingskoder OppdragK	OR
1.2	Bestilling fra selskap	Skrevet sjekkliste	OR
1.3	Køsystemet	Lagt inn litt teknisk informasjon om Kø-systemet	FS, OR
1.4	Sentralisert logging	Justert og utdypet hva og hvordan det skal logges til sentralisert logg.	EA
1.5	Køsystemet	Detaljert teknisk dokumentasjon.	OR
1.6	sjekklisten	Lagt til punkt opprettOppdrag url	OR
1.7	sjekklisten	Rettet url til opprettOppdrag og lagt til punkt om avtaler i sjekklisten	OR
1.7	publiseringsrutiner	Opprette nytt kapittel	OR
1.8	Alle	Rettet opp i fht innspill fra møte med Gjensidige	Kent Stuler
1.9	Alle	Forbedret tekster og lagt inn mangler fra kommentarer i #100.	ASO
1.10	Alle	Forbedringer etter innspill fra Gjensidige	Kent Stuler, OL og ASO
1.11	3.3.6	Opprette nytt kapittel	ASO og OL
1.12	3.3.6	Endre diagram og tekst etter innspill fra Gjensidige	Kent Stuler og OL
1.13	3.2 og 6.1	Tjeneste SendOppdrag er blocking både ved melding 110 og 111 iht. JIRA sak DBS-128. Dokumentasjon vedlikeholdes i Confluence, ProjectPlace er faset ut.	OL
1.14	Kap. 2	Endring i IP adressene og url til testmiljø ifm. migrering til ny driftsleverandør Atea. Endret også fra OppdragK til OppdrGal ifm implementering av DBS-I 20.10.2018.	OL
1.15	2.2 3.3.7	Eget pkt. i sjekkliste ved bruk av REST API/ tjenestene med token basert autentisering Beskrivelse av token basert autentisering	OL

INNHOILDSFORTEGNELSE

1	Introduksjon	5
1.1	Overordnet om dokumentet	5
1.2	Målgruppe	5
1.3	Fagsystemer i DBS	5
1.4	Blocking vs Non-Blocking og time out ved tjenestekall.....	5
2	Bestilling av integrasjon for selskap	6
2.1	Bestillingsskjema	6
2.2	Sjekkliste når et helt nytt selskap skal abonnere på webtjenester og API.....	6
3	Integrasjoner	7
3.1	Overordnet design.....	7
3.2	Ekspoonerte tjenester	8
3.3	Hovedprinsipper	9
3.3.1	Versjoner av tjenester	9
3.3.2	Toveis SSL	10
3.3.3	Validering av forespørsler/svar (XSD-validering)	11
3.3.4	Time out - generelt.....	11
3.3.5	Feilsituasjoner	11
3.3.6	Bruk av 3e-partsystem.....	12
3.3.7	Token basert autentisering	13
4	Køsystemet.....	14
4.1	Systemoversikt	14
4.2	Kø, strukturelt flytskjema	15
4.2.1	Returkoder (retur i flytskjema over) som mottas fra selskapene og håndtering- forklaring til bildet over	16
4.3	Konfigurering.....	17
4.3.1	Mottakere av email fra køsystemet	18
4.4	Duplikatkontroll.....	18
4.4.1	Duplikatkontroll slik det er implementert i DBS.....	18
4.5	Køtabell.....	18
4.5.1	Timeout for tjenester som går via kø	20
4.5.2	Logging.....	20
4.5.3	Feilhåndtering av meldinger i kø	20

4.5.4	Rydding i køtabell	21
5	Sentralisert logging i RayGun	22
5.1	Føringer for bruk av Raygun	22
5.1.1	Meldingstyper.....	22
5.1.2	Meldingsinnhold.....	22
5.2	Tags.....	24
5.3	Custom data	24
5.3.1	Sensitiv informasjon	25
5.4	Logging i produksjonsmiljø.....	25
5.5	Eksempler for logging.....	25
5.5.1	DBS til fagsystem når både request og reply logges	25
6	Referanser	27
6.1	Dokumenter	27
6.2	Tjenestekontrakter.....	27

1 Introduksjon

1.1 Overordnet om dokumentet

Dokumentet beskriver hvordan man skal integrere seg mot DBS sine tjenester, samt hvordan DBS skal integrere seg med fagsystemene hos de forskjellige selskapene.

Dette dokumentet vedlikeholdes av EVRY.

PDF-kopi er tilgjengelig på <https://produksjon.webdb.no/Kontrakter/Kontrakter.htm>

1.2 Målgruppe

Dette dokumentet skal leses av alle som skal forstå og utvikle integrasjoner mellom DBS og selskapenes fagsystemer. Det er viktig at de faglige ressursene som er superbrukere i DBS hos selskapene involveres slik at kunnskap om innholdet kan ivaretas.

1.3 Fagsystemer i DBS

DBS er i hovedsak delt opp i 2 undersystemer DBS og DBS-I.

DBS benytter følgende tjenester hos selskapene: HentFagdata, Fakturagrunnlag, Oppdrag, Fakturagrunnlag frilanser og Fakturagrunnlag tjenester verskted. DBS-I benytter tjenesten Oppdragl.

1.4 Blocking vs Non-Blocking og time out ved tjenestekall

Angir om en handling i skjermbildet blokkerer brukeren for videre arbeid inntil programmet har fått svar fra forespurt system f.eks. selskapets webservice.

Blocking:

- En modal venteboks dukker opp mens sendende system venter på svar. Bruker kan ikke fortsette inntil at svar kommer tilbake i et tjenestekall. Dersom tjenestekallet går til timeout må bruker prøve på nytt igjen. Dersom det oppstår en feil i fagsystem vises en melding i en egen meldingsboks i skjermbildet med opplysning fra selskapet. Dvs når returkode ikke er 1 (ok).
- Timeout: 10 sekunder

Non-Blocking:

- Handlingen oppleves asynkron for brukeren og blokkerer ikke i skjermbildet. Selve forespørselen blir lagt på en kø i DBS og behandlet synkront. Dersom tjenestekallet går til timeout håndterer køsystemet dette og prøver å sende på nytt.
- Timeout: 10 sekunder for statusmeldinger og 30 sekunder for oppdrag.

2 Bestilling av integrasjon for selskap

Dette kapitlet skal beskrive hvordan selskapene skal bestille tilgang mellom egne fagsystemer og til DBS-løsningen. Gjelder førstegangsbestilling, men deler av sjekkliste nedenfor kan/må utføres for hvert nytt testmiljø

2.1 Bestillingsskjema

Selskapet fyller ut bestillingsskjema og returnerer det til FNF.

Last ned skjema fra <https://produksjon.webdbs.no/Kontrakter/BestillingWebServiceTilganger.docx>

Når FNF og EVERY har gjort de nødvendige forberedelser, avtales en tid der EVERY og selskapet sammen setter opp og tester forbindelsen.

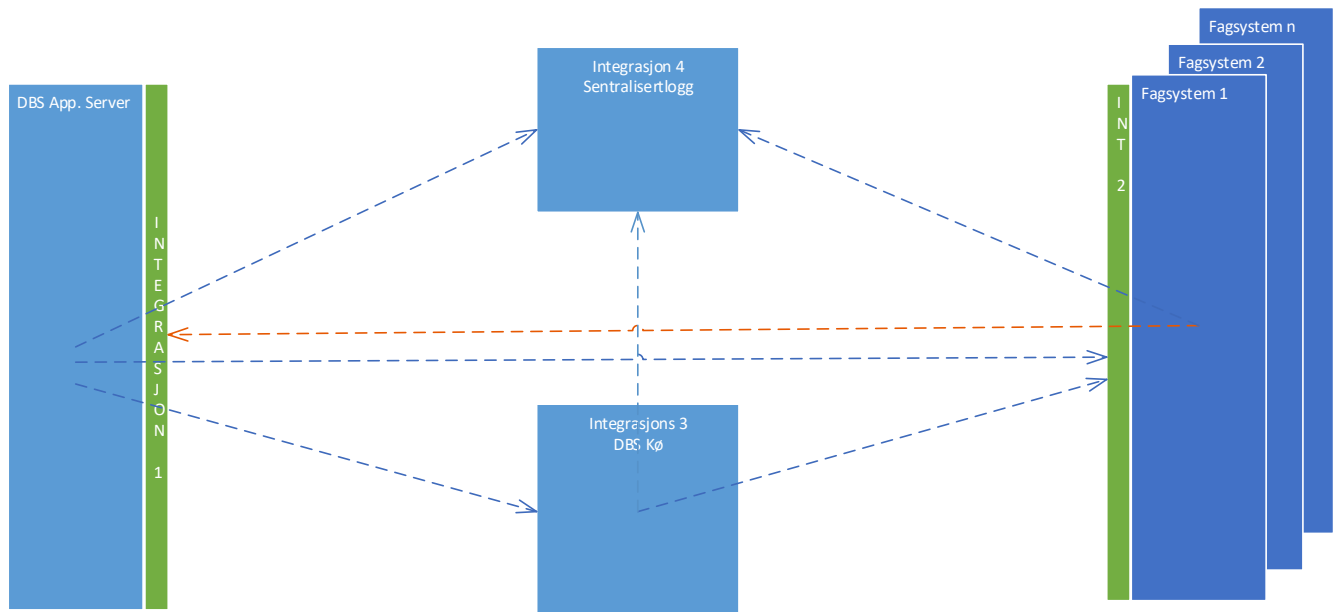
2.2 Sjekkliste når et helt nytt selskap skal abonnere på webtjenester og API

1. Selskapet har fylt ut bestillingsskjema
2. FNF har lagt inn url-er i bildet Selskapsdetaljer og satt aktivflagg og gjeldende versjon, eks «versjon 2»
3. EVERY har lagt inn e-postadresser i RayGun for det aktuelle selskapet
4. EVERY har aktivisert selskapet i DBS (databaseendringer)
5. Selskapet har fått IP-adressen til de aktuelle DBS-servere: 88.151.164.3. Alle miljøene benytter samme IP utad.
6. Selskaper som benytter OpprettOppdrag skal gå mot den url som ligger i Selskapsdetaljer for det aktuelle miljøet. Eksempler:
 - Test: <https://services-test.webdbs.no/DBSWSOpprettOppdrag/DBSWSOpprettOppdrag.svc>
 - Produksjon: <https://services-produksjon.webdbs.no/OpprettOppdrag/DBSWSOpprettOppdrag.svc>
7. Selskapet konfigurerer brannmur (om nødvendig)
8. For tjenestene med 2 veis SSL autentisering:
 - Selskapet har fått passordet til SSL Sertifikat DBS (passord-beskyttet ZIP-fil).
 - Selskapet har lastet ned og installert sertifikatet
 - Selskapet sender sitt public sertifikat til EVERY som en .cer fil.
 - EVERY installerer sertifikatet på server
9. For tjenestene med token basert autentisering: selskapet sender url for token uthenting, client_id, client_secret, og evt. scope til EVERY
10. Selskapet har dobbeltsjekket at siste versjoner av XSD og WSDL-filer (<https://produksjon.webdbs.no/Kontrakter/Kontrakter.htm>) er benyttet i webservicer. Det er viktig at selskapet påser at de namespace som er definert i wsdl/xsd videreføres.
11. Selskapet har fått hjelp av EVERY til å kjøre SoapUI-tester for å verifisere at selskapenes webtjenester er tilgjengelige fra DBS.
12. Selskapet må selv sørge for at det er opprettet verksted-avtaler.

3 Integrasjoner

Det er definert nye standardiserte integrasjonsgrensesnitt mellom DBS og alle de forskjellige selskapene. Det er laget WSDL'er og XSD'er som man kan benytte til å generere proxy-objekter. Se kapittel 6.2 Tjenestekontrakter for informasjon.

3.1 Overordnet design



DBS (applikasjonsserver):	Håndterer alle forespørsler til eksterne selskaper basert på selskapets ID. Eksponerer tjeneste for kall fra de forskjellige fagsystemene (se kapittel 3.2 Eksponerte tjenester).
Fagsystem:	I selskapene som håndterer alle forespørsler til DBS, samt eksponerer tjenester for kall fra DBS (se kapittel 3.2 Eksponerte tjenester).
DBS Kjøsystem:	Håndterer alle non-blocking meldinger fra DBS til selskapet, og håndterer resending og eventuelle feil.
Sentralisert logg:	Håndtering av sentralisert logg må implementeres både av DBS og fagsystemene. Håndterer logging av feil og alt som har med integrasjonene å gjøre. Dette vil si det skal logges når man sender meldinger og når man mottar meldinger (ikke nødvendigvis meldingsinnhold) på begge sider.

3.2 Eksponerte tjenester

Eksponeres av	Tjeneste	Metode	Beskrivelse
DBS	OpprettOppdrag	sendOpprettOppdragFraFag *)	Denne metoden benyttes av selskapene og blir brukt til å opprette oppdrag fra fagsystemet
Fagsystem	HentFagdata	hentKarroseriSkadeDetaljer *)	Denne metoden benyttes av DBS og blir brukt til å hente fagdata for karosserioppdrag fra fagsystemet.
	HentFagdata	hentGlassSkadeDetaljer *)	Denne metoden benyttes av DBS og blir brukt til å hente fagdata for glassoppdrag fra fagsystemet.
	Oppdrag	sendOppdrag *)	Denne metoden benyttes av DBS for å sende takst for oppdrag til fagsystemet. Denne metoden brukes også for kontroll av takst (Meldingskode = 111)
	Oppdrag	sendStatus	Denne metoden benyttes av DBS og blir brukt til å sende statusmeldinger for oppdrag til fagsystemet.
	Fakturagrunnlag	hentFakturagrunnlagStatus *)	Denne metoden benyttes av DBS og blir brukt til å hente fakturagrunnlagsdata for oppdrag fra fagsystemet.
	Fakturagrunnlag	sendFakturagrunnlag *)	Denne metoden benyttes av DBS og blir brukt til å sende fakturagrunnlagsdata for oppdrag til fagsystemet.
	Oppdragl	sendOppdragl	Denne metoden benyttes av DBS-I og blir brukt til å sende innløsningsdata for oppdraget til fagsystemet.
	Oppdragl	sendStatusl	Denne metoden benyttes av DBS-I og blir brukt til å sende status for innløsning til fagsystemet.
	DBS.Invoice	Freelancer/InvoiceDocumentation	Denne metoden benyttes av DBS og blir brukt til å sende honorar og omkostninger for frilanser til fagsystemet
	DBS.Invoice	Workshop/InvoiceDocumentation	Denne metoden benyttes av DBS og blir brukt til å sende

Eksponeeres av	Tjeneste	Metode	Beskrivelse
			fakturagrunnlagsdata for innløsnings- eller kontantoppgjør oppdrag til fagsystemet

*) markerte metoder sendes som Blocking.

3.3 Hovedprinsipper

1. Det innføres versjonering av webtjenester hvor maks to versjoner kan være i produksjon samtidig. Hvor lang frist selskapene får til å skifte til ny versjon, avklares av FNO
2. For SOAP webtjenestene innføres toveis SSL autentisering med utveksling av sertifikat mellom DBS og selskapene
3. For REST API er autentiseringen token basert, uten bruk av sertifikater.
4. Det anbefales å validere XSD ved sending og mottak
5. For meldinger som sendes via kø:
 - a. Det er viktig at kø-systemet påser at meldinger for et oppdrag sendes i riktig rekkefølge basert på et unikt referansnr
 - b. Ved eventuelle feil må videre meldinger for ett oppdrag ikke sendes. Med oppdrag menes meldinger som tilhører samme oppdragsID og versjonsnr. Dersom en tidligere versjon av et oppdrag ikke er mottatt OK hos et selskap sendes ikke senere versjoner.
 - c. Duplikatkontroll for melding er gjort mulig for å unngå feilsituasjoner og dobbeltlagring hos selskapene
 - d. Integrasjoner av Non-blocking-type er generelt ikke en del av prosess i DBS for å fullføre et oppdrag. Dette betyr at oppdragsprosessen fortsetter i DBS uten å ta hensyn til om meldinger er mottatt OK eller ikke. Hvis eksempelvis statusmeldinger mot formodning ikke sendes får brukere fortsatt oppdatere oppdraget i DBS/DBSK.
6. RayGun er innført som sentralt loggesystem basert på en skyløsning.
 - a. Under utvikling og integrasjonstest logges alle request og reply for å enkle verifisering og analyseres av evt feil.
 - b. I Akseptansetest og Produksjon vil det kun logges når reelle feil oppstår.
 - c. Både DBS og selskap skal logge i RayGun.
7. Time out settes for å unngå for lange svarstider og raskere finne ut av kommunikasjonsproblemer med selskapets fagsystem.

3.3.1 Versjoner av tjenester

Det er innført versjonering av webtjenester for å ha mulighet til en mer fleksibel løsning for når selskapene må utføre endringer mot en endret webtjeneste. Hvilken versjon som selskapet bruker velges fra Selskapsdetaljer, arkfane Webtjenester og koordineres med selskapene før man skifter versjon. Til tross for at versjonsnummer tilsynelatende kan velges per metode gjelder dette for webtjenesten og man må huske å ha samme versjon for alle metoder tilhørende tjenesten.

De nye standardiserte tjenestene har alle Versjon 2 mens de standardiserte tjenestene som ble utviklet våren 2015 og brukes via infotorg er versjon 1 og gjelder da HentFagdata og Fakturagrunnlag. Versjon 1 brukes av Frende men skal fases ut. Neste versjon blir da Versjon 3.

Generelt gjelder at det skal lages en ny versjon når det krever en ny generering av service hos selskap og i DBS. Ved disse type endringer i XSD skal namespace byttes ut sammen med versjonen i Selskapsdetaljer. Versjon fra tabell over webtjenester sier ikke hvilken versjon som er gjeldende av de to men angir hvor siste versjon er tilgjengelig fra: Se eksempler <https://produksjon.webdb.no/Kontrakter/Kontrakter.htm> som viser datoen i namespace i kolonne Versjon, eks: namespace="http://www.webdb.no/DBS/**2015-06-10**/DBSFakturagrunnlag.xsd.

Er endringen av en art som ikke medfører valideringsfeil kan man velge å gjøre endringen uten å lage en ny versjon. Eksempelvis: Kommentarer i XSD

Merk at i utviklings og test fasen kan det noen ganger skje at det gjøres endringer på XSD uten å lage nye versjoner. Alle som bruker den webtjeneste som endre får da melding om å generere service på nytt.

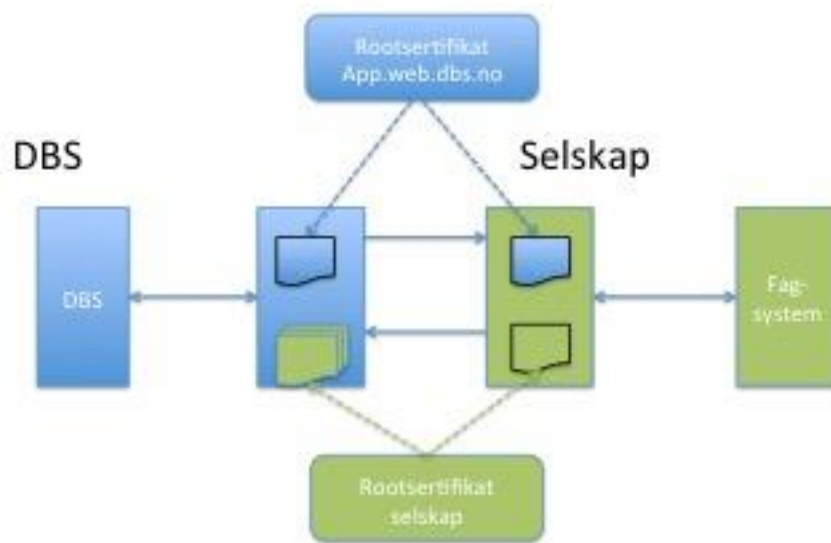
3.3.2 Toveis SSL

All informasjon som går mellom DBS og fagsystemene skal benytte Toveis SSL. Toveis autentisering refererer til mekanisme med en protokoll hvor klient og server godkjenner identitet til hverandre via gyldige sertifikater og utveksler kryptert informasjon. Følgende trinn viser hvordan toveis autentisering kan etableres mellom klient og server:

1. Klient ber om tilgang til beskyttet ressurs.
2. Serveren presenterer sitt sertifikat til klient.
3. Klient kontrollerer serveren sertifikat.
4. Hvis gyldig sertifikat, så klient sender sitt sertifikat til serveren.
5. Serveren kontrollerer klient sertifikat.
6. Hvis gyldig sertifikat, så serveren gi tilgang til beskyttet ressurser etterspurt av klient.

Alle tjenester i DBS skal være beskyttet og autentisering skal være påkrevd til å få tilgang til ressurser.

To sertifikater må bestilles (Server og klient) og SSL må aktiveres på server. Server sertifikat må brukes ved aktivering av SSL på server delen. Begge bør være public root sertifikat men DBS kan også tilby intermediate-sertifikat ved behov. Forsikringsselskapene må autentisere seg til server (DBS) ved å sende gyldig klient sertifikat. Hvis sertifikat er ikke gyldig eller et feil sertifikat har blitt sendt til server (DBS), så skal de ikke få tilgang.



Rootsertifikat selskap brukes ved tjenester hvor det er selskapet som selv initierer kallet, eks OpprettOppdrag.

Da sertifikater har en utløpsdato er det viktig at alle parter passer på å fornye sine sertifikater og gi beskjed til berørte parter om oppdatert sertifikat. Public sertifikat til DBS finnes på fellesite, se 6.2 Tjenestekontrakter

3.3.3 Validering av forespørsler/svar (XSD-validering)

Ved alle kall til tjenester så er det anbefalt at man gjør en skjema kontroll basert på informasjon definert i gjeldende XSD-skjema. Både i forhold til forespørsel og svar, slik at man kan garantere for at informasjon som blir utvekslet er på riktig format. Dette i sin tur krever at man har et forhold til grupper og felt som har occurs min=0 og occurs min<>0.

3.3.4 Time out - generelt

Selskapene anmodes å sette en intern time out som er 2 sekunder lavere enn time out i DBS, slik at det enklere kan avdekkes om time out kommer fra kommunikasjon mellom DBS og selskapet eller internt fra selskapet. Det vil skje en tilpasning av time out for de enkelte tjenester som gir oss mest mulig stabile svarstider.

Time out kan settes på tjenestenivå (anbefales) og på selskapsnivå. Liste over standard time out finnes her: <https://produksjon.webdbs.no/Kontrakter/Kontrakter.htm>

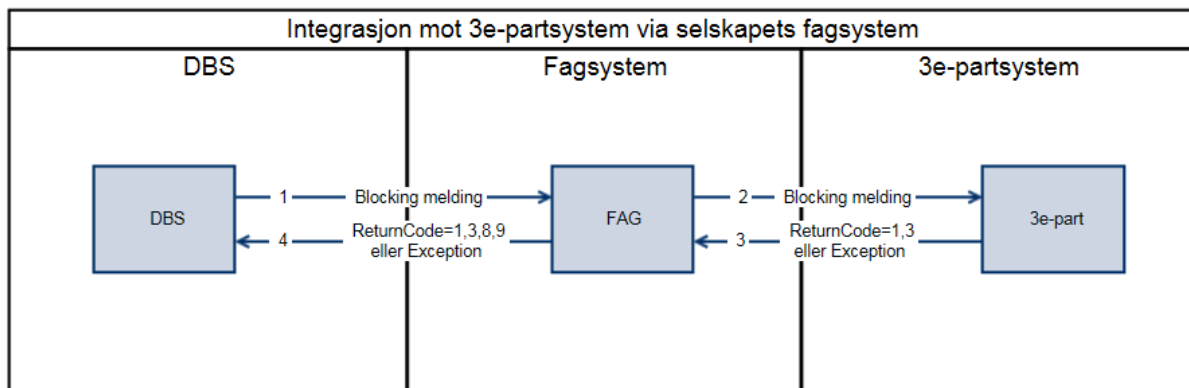
3.3.5 Feilsituasjoner

Ved forespørsler og svar til/fra en tjeneste, så skal det ikke fra DBS kastes en Exception gjennom integrasjonen. Dersom kommunikasjon er opprettet skal det logges, og håndteres på den siden feilen har oppstått. Det skal ikke sendes noen form for Exception-melding. Tjenesten skal i stedet bruke ReturnCode og ReturnMessage for å gi tilbakemelding på at noe har feilet.

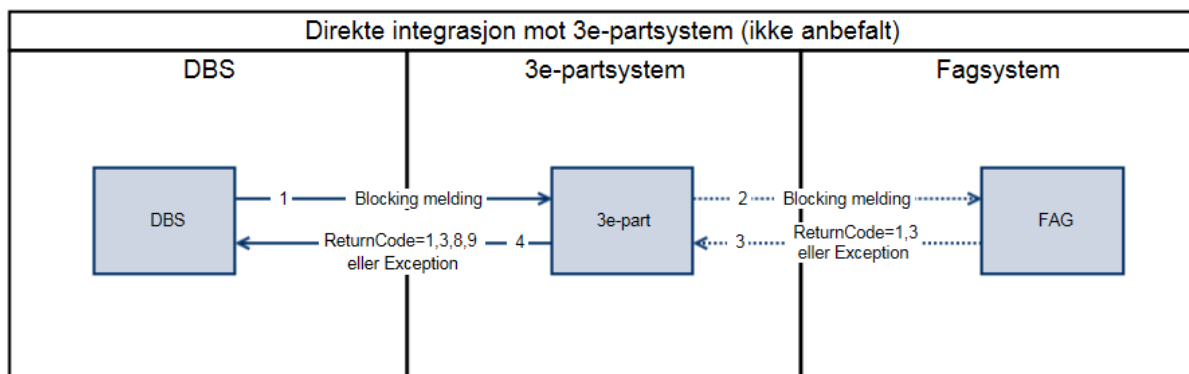
3.3.6 Bruk av 3e-partsystem

Ved integrasjon mot 3e-part anbefales at selskapet selv håndterer trafikken mot 3e-part og at DBS ikke kommuniserer direkte med 3e-partsystem. Da er det selskapet som eier prosessen og selv kan bestemme når og hvor meldingen skal lagres eller videresendes.

Dersom selskap benytter seg av 3e-part som direkte mottak av meldinger fra DBS ved kontroll av takst, har ikke lenger DBS kontroll at selskap får informasjon om taksten. Melding fra DBS som er av type Blocking, bør sendes til fagsystem fra 3e-part som type Blocking. Dette for å sikre at selskapet også har fått opplysning om oppdraget ettersom et positivt svar fra kontroll av takst gjør at prosessen går videre i DBS. Det kan ellers oppstå problemer i mottak av statusmeldinger fra DBS i fagsystemet, fordi statusmeldingene sendes via kø uten å vite om selskapet har mottatt oppdragsinfo eller ikke.



Ved direkte integrasjon mot 3e-part kan det oppstå tilfeller av manglende informasjon om oppdraget i fagsystemet. Eks. 3e-part mottakstjeneste feiler og meldinger til fagsystem blir ikke sendt. I dette tilfelle må fagsystemet hente oppdragsdata fra DBS sitt datavarehus eller 3e-part kontaktes for videre overføring av info.



Følgende punkter bør avklares ved direkte integrasjoner mot 3e-part:

- Hva leverer 3e-part til fagsystemet?
- Når sendes oppdragsinfo til fagsystemet? I hvilken rekkefølge?

- Venter 3e-part på svar fra fagsystemet før den responderer DBS? Hvis ja, bør det gjøres grundige tester med svartider. Hvis ikke må konsekvenser utredes i forhold til prosessen i DBS.

3.3.7 Token basert autentisering

Autentiseringen hostes hos forsikringsselskap og er bygget på [OAuth 2.0](#)

1. For å aksessere API må klienten forespørre et access_token fra autorisasjonsserveren. Klienten gjør en POST-forespørsel med Content-Type = "application/x-www-form-urlencoded" i header, og inneholder:
 - grant_type = "client_credentials"
 - client_id = (utstedes av selskapet)
 - client_secret = (utstedes av selskapet)
 - scope = (utstedes av selskapet, hvis ønskelig)
2. Autorisasjonsserveren mottar tokenforespørselen og utfører en klientautentisering.
3. Dersom den autentiserte klienten har tilgang til de forespurte ressursene returneres et access_token til klienten. Responsen kommer med Content-Type = "application/json", og inneholder:
 - access_token = "WblfKdiRLJkZtJFMT0="
 - expires_in = 300
 - token_type = "bearer"
4. Klienten kan nå aksessere den ønskede ressursen ved bruk av access_tokenet.
5. Token levetid er 15 min. Klienten ber om nytt token ved hvert kall mot API.

4 Køsystemet

Køsystemet består både av en køtjeneste, og en køtabell. Tjenesten er en standard Windows-tjeneste som kjører på AppServer og har navnet DBS.Queue.WindowsService. Køtabellen er en SQL-tabell som heter Que og inneholder informasjon om meldingen som sendes, inkludert XML.

Tjenesten skanner køtabellen kontinuerlig og sender en og en melding til selskapet det gjelder. DBS, og DBS-K legger meldinger som skal sendes i køtabellen.

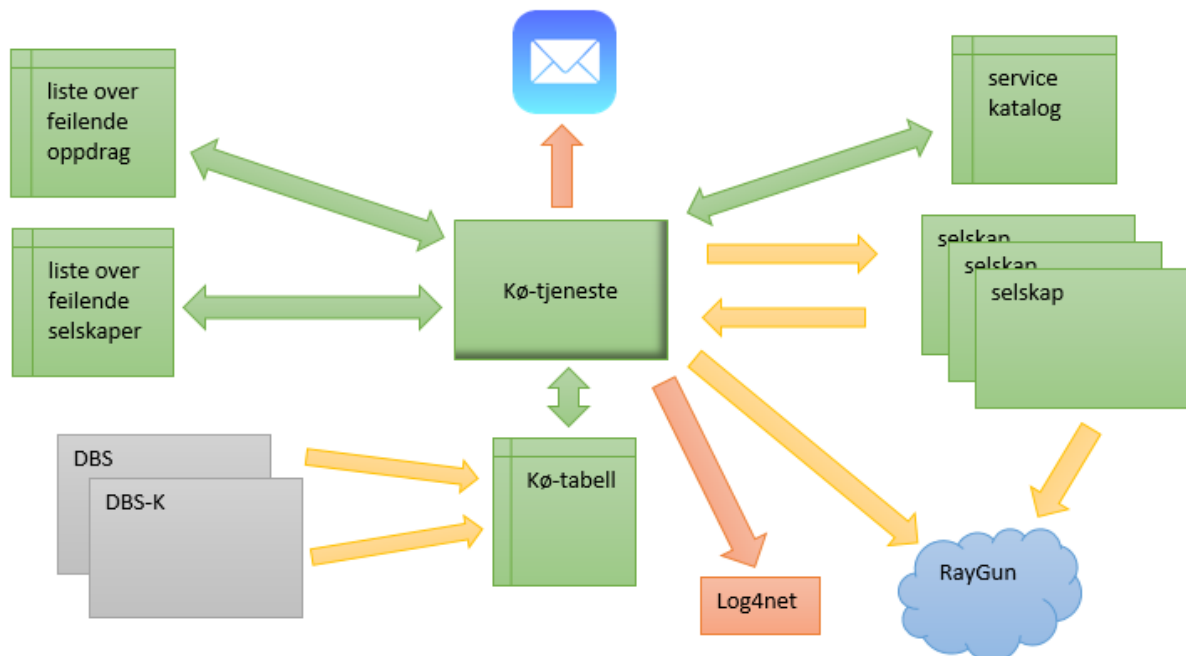
For hvert gjennomløp av tabellen bygges en liste over feilende oppdrag og en liste over selskaper med kommunikasjonsproblemer i en skip-liste. Disse listene tømmes av tjenesten før hvert gjennomløp.

En servicekatalog inneholder en oversikt over hvilke servicer hvert selskap støtter og om disse er aktive. Denne katalogen vedlikeholdes fra DBS-klienten i Selskapsdetaljer/Webtjenester. Det er kun de selskapene som har aktiv bruk av de aktuelle non-blocking metodene som blir rutet via kø. Køtjenesten logger da alle meldinger som sendes, hvilke svar som returneres og eventuelle feil i forbindelse med meldingsutveksling på den aktuelle metoden.

Køsystemet brukes av webtjenestene Oppdrag og OppdragI, og sørger for en enhetlig behandling av sending, resending, timeout og feilsituasjoner. Merk: Kontroll av takst som også benytter seg av metode SendOppdrag går ikke via kø ettersom prosessen for å gå videre til manuell kontroll/autogodkjenning krever svar fra selskapet.

Køpostene/statuspostene trigges ved ajourføring av oppdragene i DBS og DBS-I, og finnes på 4 formater: SendStatus, SendOppdrag, SendStatusI og SendOppdragI.

4.1 Systemoversikt

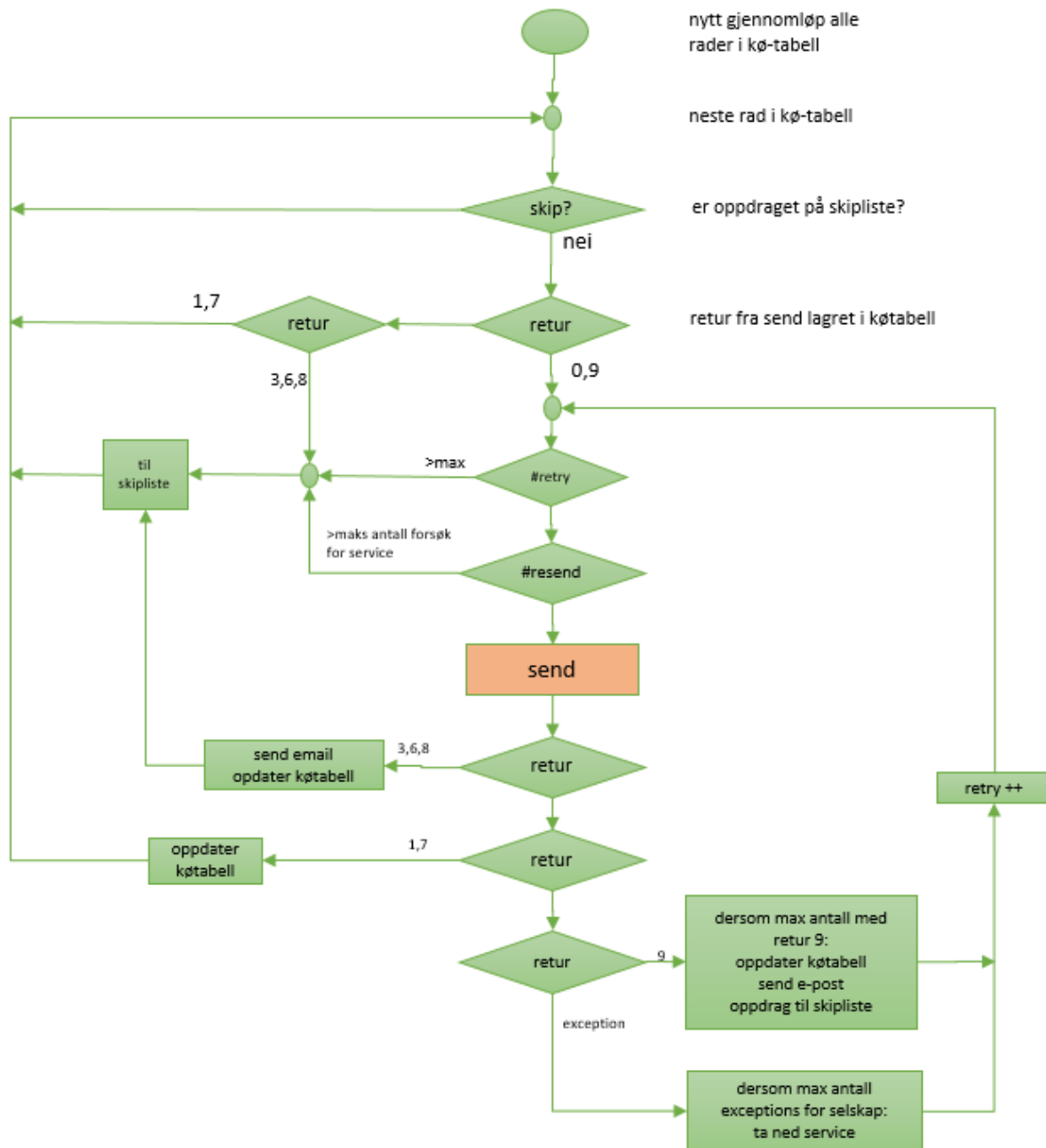


Grønn: data til og fra databaseserver eller tabeller i minnet. Gul: meldingsinnhold. Orange: informasjon om feil

4.2 Kø, strukturelt flytskjema

Flytskjemaet beskriver forenklet logikk i køsystemet. Tjenesten løper gjennom køtabellen kontinuerlig og sender meldinger når forutsetningene er til stede. Utplukkskriterier for gruppering av melding til sending: Selskapsnr, Oppdragsnummer samt Referansenummer sortert på laveste referansenummer først. Dette skal sikre at korrekt rekkefølge sendes og at senere versjoner av et oppdrag ikke sendes før tidligere er blitt mottatt og akseptert av selskapet.

Skip-listen (liste over feilende oppdrag) er et array i minnet som resettes for hvert gjennomløp av tabellen. Listen vil bygges opp med oppdragsnumre som av ulike årsaker ikke skal sende meldinger om til selskapene. Meldingene i skip-listen vil så bli behandlet manuelt i samråd med aktuelt selskap. Legg merke til at feilreturer fra selskapet håndteres forskjellig fra andre typer feil - Exception.



4.2.1 Returkoder (retur i flytskjema over) som mottas fra selskapene og håndtering- forklaring til bildet over

Returkode		
0/blank	Håndteres som returkode 9	
1	Alt Ok	Sender neste sak i kø
3	<i>Ikke i bruk for tjenester via kø</i>	N/A
6	Rekkefølgeproblem, eks status kommer før selve oppdraget.	Stopper videre sending til selskapet innenfor samme oppdragsid, og sender email til angitte personer i hhv QueExceptionMailTo, og QueExceptionMailCC, Se 4.3

Returkode		
7	Selskap har allerede mottatt meldingen, håndteres som 1. Se 4.4	Sender neste sak i kø
8	Selskapet avviser meldingen pga feil i data/manglende felt etc	Stopper videre sending til selskapet innenfor samme oppdragsid og sender email til angitte personer i hhv <code>QueExceptionMailTo</code> , og <code>QueExceptionMailCC</code> , se 4.3
9	Teknisk feil i fagsystem av ulike grunner (time out, ikke tilgjengelig etc)	Resender opp til maks antall ganger (<code>MaksRetry</code>). Hvis fortsatt feil stoppes videre sending til selskapet og email sendes til angitt(e) personer (<code>QueExceptionMailTo</code> , og <code>QueExceptionMailCC</code> , se 4.3

4.3 Konfigurering

Sentrale parametere i køsystemet kan endres i `app.config`. Tabellen under er en oversikt over disse innstillingene (# angir verdier brukt under testing):

Disse kan også tilpasses per selskap dersom nødvendig.

Key i <code>appSettings</code>	#	
<code>MaxRetry</code>	15	Antall ganger en melding kan feile med <code>ReturnCode=9</code> . Forhindrer uendelig looping. Når <code>MaxRetry</code> er oppbrukt slutter køen å sende og email sendes til angitt(e) personer (<code>QueExceptionMailTo</code> , og <code>QueExceptionMailCC</code> , Se 4.3
<code>NumberOfRequestsToService</code>	3	Antall ganger en feilende webtjeneste som er gått ut på time out, eller annen intern feil (Exception) blir forsøkt sendt. Selskapet som bruker tjenesten legges i liste over feilende selskaper dersom antallet overstiges. Dersom selskapet mislykkes <code>NumberOfRequestsToCompany</code> ganger, tas service kall til det aktuelle selskaper helt ned i <code>TotalDownTimeForServiceInHours</code> timer.
<code>NumberOfRequestsToCompany</code>	3	Antall feilende meldinger som sendes til selskap som aksepteres før aktuell service legges i karantene i <code>TotalDownTimeForServiceInHours</code> timer.
<code>TotalDownTimeForServiceInHours</code>	1	Etter <code>TotalDownTimeForServiceInHours</code> timer blir service igjen tilgjengelig. Dersom <code>MaxRetry</code> er passert, blir likevel ikke meldingen sendt.
<code>NumberOfRequestsOnReturnCode9</code>	3	Antall ganger en melding kan feile på selskapssiden (returkode=9) før epost sendes og melding legges på skipliste i køtabellen – Ikke i bruk, p.t
<code>TimeOutForSendStatusInSeconds</code>	10	Timeout for metode <code>SendStatus</code> . Angis i antall sekunder for hvor lenge køsystem venter på svar fra selskapet før timeout exception.
<code>TimeOutForSendEstimateInSeconds</code>	30	Timeout for metode <code>SendOppdrag</code> . Angis i antall sekunder for hvor lenge DBS/køsystem venter på svar fra selskapet før timeout exception.

Key i appSettings	#	
TimeOutForSendStatusKInSeconds	10	Timeout for metode SendStatusI. Angis i antall sekunder for hvor lenge køsystem venter på svar fra selskapet før timeout exception.
TimeOutForSendEstimateKInSeconds	30	Antall sekunder DBS kan vente på svar på sendOppdragI-melding før det kastes en timeout exception.
QueExceptionMailTo		Mottakere av mail sendt på grunn av returkode 3,6,8,9 eller exception. Se 4.3.1
QueExceptionMailCC		CC (se QueExceptionMailTo)
QueExceptionMailFrom		Avsender (se QueExceptionMailTo)

4.3.1 Mottakere av email fra køsystemet

Dette er i første omgang tenkt å være til ressurser hos EVRY AM. Men dersom ønskelig er det mulig at også teknisk personell hos selskapene kan varsles direkte fra køen. Men det må da opprettes en konfigurering for det aktuelle selskapet. Selskapene anbefales i å benytte seg av RayGun men der logges ikke feil i køen annet enn når det ikke i det hele tatt er mulig å legge en melding i køen.

4.4 Duplikatkontroll

Det er lagt opp til at alle aktuelle non-blocking tjenester kan implementere duplikatkontroll slik at samme melding ikke behandles av mottakende fagsystem flere ganger.

4.4.1 Duplikatkontroll slik det er implementert i DBS

Det er blitt opprettet to felt i de metoder som er aktuelle for duplikatkontroll og som brukes ved kontroll i mottakende system:

- Referansenummer – unikt referansenummer til meldingen, og er unikt i det miljø den opprettes i
- Timestamp av type datetime som angir når meldingen første gang ble prøvd sendt. Denne er kun til info til mottakende system og er i sin natur ikke unik ettersom formatet ikke har lavere nivå enn sekunder.

4.5 Køtabell

Kø-tabellen Que (queue er et reservert navn og kan ikke benyttes) opprettes i egen database og inneholder all nødvendig informasjon om meldingene som sendes til/mottas fra fagsystemene. Selskapene har p.t en felles køtabell.

Køtabellen er en nyttig kilde for feilsøking fordi all historikk forblir i tabellen. Det er ingen automatikk for rydding i tabellen. Ved behov kan rader manuelt flyttes over i en historikktabell. Det finnes planer om å tilby et verktøy for overvåking av kø. Kø-tabellen vil bli sentral i en slik løsning. En detaljert beskrivelse av feltene i kø-tabellen er listet under.

	QueueId	IsSent	CompanyNumber	EstimateNumber	EstimateVersion	LicenseNumber	Request	RequestSentDate	ReTry	ReturnCode	ReturnText	CurrentUser	CurrentUserDate	QueDate	MessageCode	WebserviceTypeid	WebserviceVersion
1	1600	1	0099	15800931	0	LJ51820	<?xml version="1.0"?> <sendOp...	2015-11-02 02:27:49...	0	1	OK, mottak bekreftes.	DBS01B1	2015-11-02 02:27:36...	2015-11-02 02:27:36...	100	80	2
2	1601	1	0099	15800931	0	LJ51820	<?xml version="1.0"?> <sendSt...	2015-11-02 02:27:52...	0	1	OK, mottak bekreftes. Je...	DBS01B1	2015-11-02 02:27:36...	2015-11-02 02:27:36...	120	70	2
3	1602	1	0099	15800931	0	LJ51820	<?xml version="1.0"?> <sendSt...	2015-11-02 02:28:56...	0	1	OK, mottak bekreftes. Je...	DBS01B1	2015-11-02 02:28:48...	2015-11-02 02:28:48...	130	70	2
4	1603	1	0099	15800931	0	LJ51820	<?xml version="1.0"?> <sendSt...	2015-11-02 02:33:10...	0	1	OK, mottak bekreftes. Je...	DBS01B1	2015-11-02 02:33:05...	2015-11-02 02:33:05...	140	70	2
5	1604	1	0099	15800931	0	LJ51820	<?xml version="1.0"?> <sendSt...	2015-11-02 02:34:15...	0	1	OK, mottak bekreftes.	DBS01B1	2015-11-02 02:34:04...	2015-11-02 02:34:04...	300	90	2
6	1605	1	0099	15800931	0	LJ51820	<?xml version="1.0"?> <sendSt...	2015-11-02 02:34:18...	0	1	OK, mottak bekreftes. Je...	DBS01B1	2015-11-02 02:34:06...	2015-11-02 02:34:07...	180	70	2
7	1606	1	0099	15800931	0	LJ51820	<?xml version="1.0"?> <sendOp...	2015-11-02 02:34:19...	0	1	OK, mottak bekreftes.	DBS01B1	2015-11-02 02:34:07...	2015-11-02 02:34:07...	190	80	2
8	1607	1	0099	15800931	0	LJ51820	<?xml version="1.0"?> <sendSt...	2015-11-02 02:34:52...	0	1	OK, mottak bekreftes.	DBS01B1	2015-11-02 02:34:42...	2015-11-02 02:34:43...	310	90	2
9	1608	1	0099	15800931	0	LJ51820	<?xml version="1.0"?> <sendSt...	2015-11-02 02:35:05...	0	1	OK, mottak bekreftes.	DBS01B1	2015-11-02 02:35:00...	2015-11-02 02:35:00...	320	90	2
10	1609	1	0099	15800931	0	LJ51820	<?xml version="1.0"?> <sendSt...	2015-11-02 02:35:29...	0	1	OK, mottak bekreftes.	DBS01B1	2015-11-02 02:35:23...	2015-11-02 02:35:23...	330	90	2

Figur: eksempel på utlisting. For beskrivelse av feltene, se nedenfor

Felt	Innhold
QueueId	Unik nøkkel (en fortløpende teller). Dette er referansenummeret som sendes fagsystemet
IsSent	En bryter, 1 betyr at meldingen er sendt til fagsystemet 0 betyr at køen ikke har sendt ennå
CompanyNumber	Selskapsnr, som bestemmer hvilken fagsystem-webtjeneste det skal sendes til. Ut ifra denne (og orgenhet 00-000), finnes webtjeneste-URL, dvs service-endpointet (fra CompanyWebService-tabellen) som igjen er lagt inn i Selskapsdetaljer/Webtjenester
EstimateNumber	DBS Oppdragsnr
EstimateVersion	Oppdragsversjon.
LicenseNumber	Registreringsnr kjøretøy
Request	Melding som sendes til selskapet i XML-format basert på aktuell XSD til metoden
RequestSentDate	Sendt dato og forteller når meldingen siste gang ble sendt
ReTry	Antall ganger køen har prøvd å sende statusposten om igjen til fagsystemet ved Exception eller feilkode 9 (dersom han ikke lykkes første gang). Køen gir opp å sende, etter et visst antall forsøk (MaxRetry, settes i config-filen, default 15 men satt til 6 i testmiljøer)
ReturnCode	Returkoden mottatt fra fagsystemet
ReturnText	Returteksten mottatt fra fagsystemet
CurrentUser	Gjeldende pålogget DBS-brukerid (som dannet meldingen)
CurrentUserDate	Tidspunktet melding ble forsøkt re-sendt. (Får ikke verdi før ved resending)
QueDate	Tidspunktet melding ble opprettet og lagt på køen. Dette feltet legges inn i felt Timestamp i meldingen i Request og endres ikke.
MessageCode	Meldingskode, 3 sifret i meldingen, eks 110, 165, 190 etc
WebserviceTypeid	Id til hvilken metode meldingen inneholder. Tabell Webservice i database Grunndata brukes som oppslag

Felt	Innhold
WebserviceVersion	Hvilken versjon av tjenesten som ble brukt og som hentes fra Selskapsdetaljer på det aktuelle tidspunktet.

4.5.1 Timeout for tjenester som går via kø

Timeout pr tjeneste er default på 10 eller 30 sekunder men kan settes individuelt på flere nivåer ved behov. Disse nøklene i app.config definerer timeout: TimeOutForSendStatusInSeconds, TimeOutForSendEstimateInSeconds, TimeOutForSendStatusKInSeconds, TimeOutForSendEstimateKInSeconds. Brudd på timeout setter i gang feilhåndteringsmekanismer i køen. Se 4.5.3

4.5.2 Logging

Alle meldinger som sendes og mottas i kø-systemet og selskapenes servicer logges til RayGun under utvikling og fram til Akseptansetest. Logger fra kø-systemet kan se slik ut i RayGun dashboard:

☐	Exception: <u>Oppdrag_sendStatus_DBS_GotResponse_15800769-0</u>
☐	Exception: Oppdrag_sendStatus_DBS_Mock_SendingResponse_15800769-0
☐	Exception: <u>Oppdrag_sendStatus_DBS_SendingRequest_15800769-0</u>
☐	Exception: <u>Oppdrag_sendOppdrag_DBS_GotResponse_15800769-0</u>
☐	Exception: Oppdrag_sendOppdrag_DBS_Mock_SendingResponse_15800769-0
☐	Exception: <u>Oppdrag_sendOppdrag_DBS_SendingRequest_15800769-0</u>

I tillegg logger køen også vha Log4Net (for intern logging av større datamengder til flate filer) og EventViewer (for logging av mer alvorlige teknisk interne feilmeldinger). Request/Response logges i Log4Net for ekstra kontroll over hva som er blitt sent/mottatt. I tillegg brukes RayGun som sentral logging, se 5

4.5.3 Feilhåndtering av meldinger i kø

- Ettersom respons som mottas fra selskaper blir lagret i en tabell i databasen er det enklere å finne ut hva skjedde med meldingen.
- Hvis en transaksjon feiler (returkode 9 eller timeout), så skal køsystemet prøve å sende samme oppdrag opp til antall ganger som er konfigurert. Hvis flere oppdrag feiler mot samme selskap, så skal tjenesten gå ned automatisk. Det sendes så varsel via Epost til forhåndsdefinerte personer at tjenesten er tatt ned for selskapet.
- Hvis en transaksjon feiler med andre returkoder fra selskapet prøver EVERY AM å sjekke om det er mulig å rette opp direkte i XML for å få meldingen sendt. Avhengig av hvorfor dette avvirket oppsto settes det også i gang andre tiltak som skal gjøre at det ikke skjer igjen. Dersom det ikke er mulig å gjøre noe med XML tar EVERY AM kontakt med selskapet for å finne en løsning på problemet.

4.5.4 Rydding i køtabell

Da køtabellen vil vokse veldig rask må denne tømmes ved jevne intervaller. Det vil kjøres ryddejobber som vil slette alle meldinger som er eldre en 6 måneder.

5 Sentralisert logging i RayGun

RayGun er et loggesystem som er skybasert og skal brukes ved integrasjon mellom selskap/DBS og vise versa for å fange opp feilsituasjoner og forhåpentlig gjøre det enklere å finne årsaker til feil. Det er ikke tiltenkt at RayGun skal benyttes til logging av interne feil i DBS og/eller fagsystemene.

Hvert selskap får sine egne så kalte Applikasjoner en for test og en for produksjon. Tilganger bestilles av Bilskadekontoret.

Under utvikling og test skal alle meldinger som sendes logges og kan brukes ved verifisering av korrekthet i XML vs DBS og ved feilsøking.

I Produksjon logges det kun feil men det er også mulig til tider å slå på logging ved behov der. Se kapittel 5.4 Logging i produksjonsmiljø

For informasjon om RayGun, se:

- Hjemmeside: <https://raygun.io/>
- Dokumentasjon: <https://raygun.io/docs>
- Bibliotek: <https://raygun.io/raygun-providers>

5.1 Føringer for bruk av Raygun

Når man logger en melding så skal den logges på 2 forskjellige måter, basert på om det er oppstått en feil eller ikke. I tillegg er det påkrevd at det på hver side opprettes mulighet for å skru av/på mulighet til å logge på meldingstypenivå.

5.1.1 Meldingstyper

Det legges i første omgang opp til at sentralisert logg kan/skal logge 3 typer:

- Error - Når det er oppstått en exception.
- Info - Når en melding er sendt eller mottatt.
- Debug - Utfyllende melding om hva som er sendt og/eller mottatt.

I produksjon skal primært meldingstype 'Error' logges. I testmiljøer skal meldingstype 'Info' og 'Debug' brukes for å forenkle feilsøking om det skulle oppstå en feil.

5.1.2 Meldingsinnhold

Raygun er i utgangspunktet et feilloggingssystem, og selv om Raygun har mulighet for å opprette eget meldingsobjekt, er det besluttet at vi benytter feilobjekt Error for all logging (dette fordi det gir litt ekstra informasjon på loggmeldingene).

Meldingstypen bestemmer hva meldingsinnholdet skal være. I tillegg skal meldingene inneholde tags for enklere å kunne spore (se [Tags](#)).

For en eller flere av meldingstypene er det også aktuelt å logge innholdet i request- og response-objektene, dette skal legges inn på loggmeldingen som «custom data» (se [Custom data](#)).

Kommunikasjon mellom DBS og de enkelte selskapene skal logges på egne logger, kalt applications i Raygun. Hvert (aktuelt) selskap vil få tildelt nøkler (AppKey) som brukes for logging til aktuell logg – en nøkkel som brukes for testmiljøer og en nøkkel som skal brukes i produksjonsmiljøet. Aktuell AppKey finner du til venstre i menyvalg ApplicationSettings og under General Settings.

5.1.2.1 Meldingsinnhold – Error

For meldingstype Error logges oppstått Exception til sentralisert logg, samt tags og eventuelt informasjon om request/response (custom data).

Meldingstypen error skal fortrinnsvis benyttes når det har oppstått en feil knyttet til kommunikasjonen mellom selskapenes systemer og DBS. Men under utvikling og test logges alle meldinger for enklere å finne hva som sendes og mottas.

Eks. (.net/C#):

```
var client = new RaygunClient(appKey);
var customData = new Dictionary<string, object>
{
    { "REQUEST", request },
    { "RESPONSE", response }
};
client.Send(exception, new[] { "Error", "15600100-0" }, customData);
```

(appKey = nøkkel til aktuell log/application i Raygun, request = aktuell service-request, response = aktuell service-response, exception = aktuell Exception, "15600100-0" = aktuelt oppdragsnr og versjon (bruker eventuelt reg.nr. dersom oppdragsnr ikke finnes)).

5.1.2.2 Meldingsinnhold – Info

For meldingstype info logges en meldingstekst til sentralisert log, samt tags.

Meldingsteksten skal ha følgende oppbygning:

<tjeneste>_<metode>_<side>_<handling>

Eks.: Fakturagrunnlag_sendFakturagrunnlag_DBS_SendingRequest

«Side» beskriver hvem som logger og skal være en av følgende: Selskapsnavn, eller DBS

«Handling» vil typisk være:

- SendingRequest
- GotRequest
- SendingResponse
- GotResponse

5.1.2.3 Meldingsinnhold – Debug

Meldingstypen debug er tenkt brukt i forbindelse med oppkobling av nye selskaper, samt ved feilsøking i eventuelle feilsituasjoner.

For meldingstype debug logges en meldingstekst til sentralisert log, tags og informasjon om service-request og -response (custom data).

Meldingsteksten skal ha følgende oppbygning:

<tjeneste>_<metode>_<side>_<handling>_<<oppdragsnummer>-<versjon> eller reg.nr.

«Side» beskriver hvem som logger og skal være en av følgende: Selskapsnavn, eller DBS

«Handling» vil typisk være:

- SendingRequest
- GotRequest

- SendingResponse
- GotResponse

Eks.: HentFagdata_hentKarosseriSkadeDetaljer_DBS_GotResponse_AB12345

Eks kode: (.net/C#):

```
var client = new RaygunClient(appKey);
var messageText = "HentFagdata_hentKarosseriSkadeDetaljer_DBS_GotResponse_AB12345";
var customData = new Dictionary<string, object>
{
    { "REQUEST", request },
    { "RESPONSE", response }
};
client.Send(new Exception(messageText), new[] { "Debug", "AB12345" }, customData);
```

(appKey = nøkkel til aktuell log/application i Raygun, request = aktuell service-request, response = aktuell service-response, "AB12345" = aktuelt reg.nr. (bruker eventuelt oppdragsnr-versjon dersom dette finnes)).

5.2 Tags

Tags legges inn på meldingen som en liste av strenger (i aktuell parameter).

Alle meldinger som sendes til loggen skal inneholde en tag som forteller hvilken meldingstype det er. Lovlige verdier er (kun en per melding):

- Error
- Info
- Debug

I tillegg skal det legges på en tag som identifiserer hvilket oppdrag meldingen gjelder, eventuelt hvilket kjøretøy dersom oppdrag ikke er opprettet enda.

- Når oppdragsnummer er del av meldingen skal dette, sammen med versjon, brukes som tag. Formatet skal være: <oppdragsnummer>-<versjon>, eks.: 15600100-0
- Dersom oppdragsnummer ikke er tilgjengelig, f.eks. ved henting av fagdata, skal registreringsnummer på aktuelt kjøretøy brukes som tag, eks.: AB12345

5.3 Custom data

Raygun gir mulighet til å lagre egendefinerte data på en loggmelding, såkalt «custom data». Det er bestemt at vi skal bruke denne parameteren til å sende med informasjon om request og response til/fra web service.

Objektet som legges til i CustomData skal inneholde en dictionary bestående av en streng og et objekt, og den skal inneholde en rad for REQUEST og en rad for RESPONSE.

Eks. (.net/C#):

```
var customData = new Dictionary<string, object>
{
    { "REQUEST", request },
    { "RESPONSE", response }
};
```

Dersom response-objekt ikke finnes enda, typisk ved SendingRequest og GotRequest, legger man inn NULL.

5.3.1 Sensitiv informasjon

Visse opplysninger kan være sensitive og skal ikke bli med i sentralisert logg. I en testfase med fiktive data er det ingen sensitive data og det er trygt å logge alt. Det er p.t ikke funnet behov av å maskere enkelte data om kunden/oppdraget.

5.4 Logging i produksjonsmiljø

Produksjonsmiljøet (hos selskapene og DBS) skal logge til egne logger per selskap (egen nøkkel tildelt hvert aktuelt selskap). Grunnen til at det skilles på logging i testmiljøer og i produksjon er at det skal være enklere å se/ oppdage feil i produksjon.

I utgangspunktet skal det i produksjon kun logges meldinger av typen Error, dette for å begrense logg-mengden (det er en månedlig maksgrense i den sentraliserte loggen). Det må likevel være mulig for selskap og DBS å skru på utvidet logging i produksjon ved behov, typisk ved eventuelle feilsituasjoner. En eventuell endring av nivå for logging i produksjon må skje i samarbeid mellom aktuelt selskap og DBS (og eventuelt FNO).

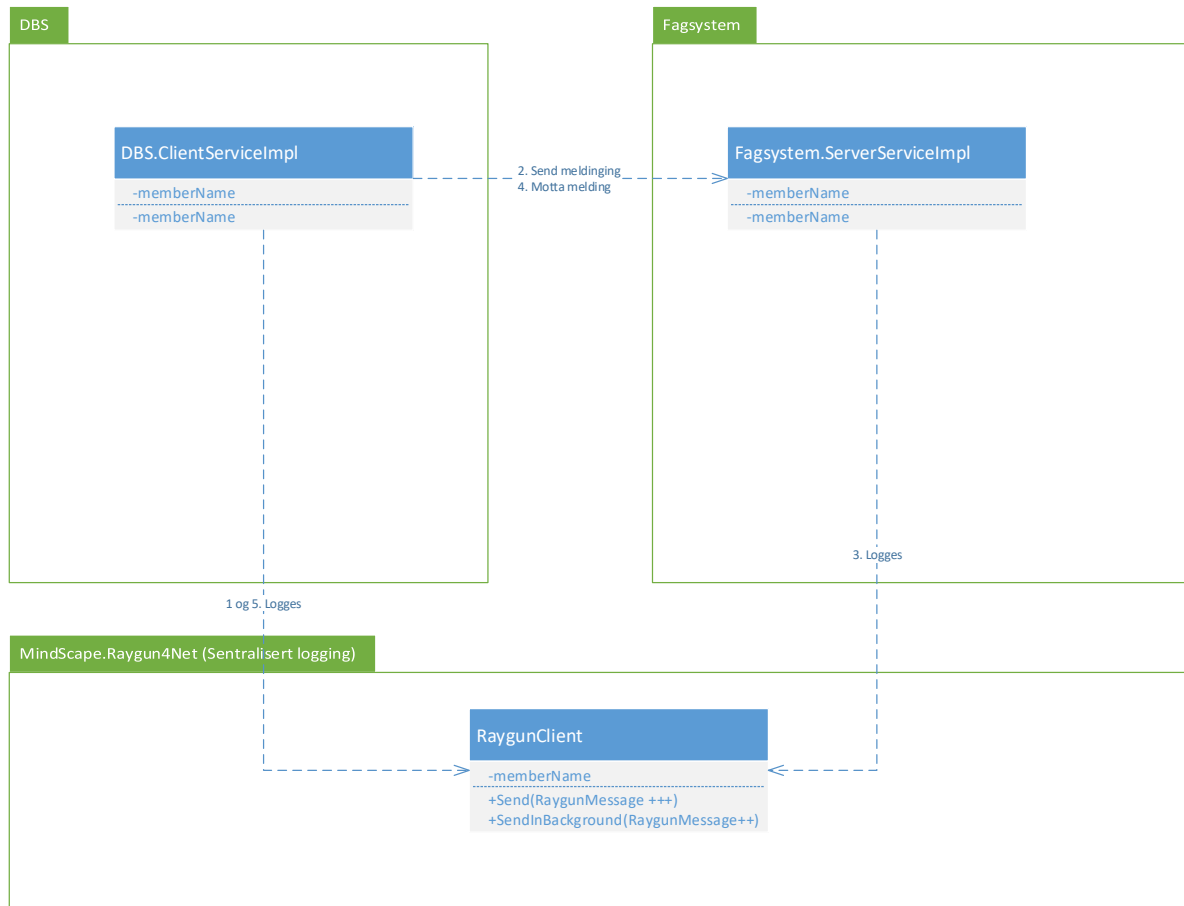
Raygun har innebygget mulighet for varsling ved registrerte feil/ meldinger, selskapene bestemmer selv om de ønsker å abonnere på dette. (Settes opp i Raygun av den enkelte bruker).

5.5 Eksempler for logging

Dette kapittelet inneholder eksempler på hvordan man skal logge.

5.5.1 DBS til fagsystem når både request og reply logges

Dette er et eksempel på hvordan logging skal foregå mellom DBS og fagsystem. Det skal også fungere på samme måte fra fagsystemene til DBS.



1. DBS: Melding logges før den sendes til fagsystem
2. DBS: Melding sendes til fagsystem
3. Fagsystem: Melding blir mottatt, informasjon oppdateres, svaret genereres og forespørsel og svar logges (kan eventuelt logge både ved mottatt forespørsel og ved sending av svar)
4. Fagsystem: Svar blir sendt tilbake til DBS
5. DBS: Melding er mottatt og logges

NB! Husk at man skal logge lokalt på samme måte som før. Dette er kun for å se meldingsflyten mellom systemene, samt gjøre det enklere å finne ut hva som har skjedd mellom systemene.

6 Referanser

6.1 Dokumenter

Referanse	Dokument
Vedlegg 1	WS - OpprettOppdrag
Vedlegg 2	WS – Fakturagrunnlag
Vedlegg 3	WS - HentFagdata
Vedlegg 4	WS - Oppdrag
Vedlegg 5	WS - Oppdragl
Vedlegg 6	API Fakturagrunnlag frilanser
Vedlegg 7	API Fakturagrunnlag tjenester

Dokumentene vedlikeholdes i Confluence av Bilskadekontoret.

PDF-kopier finnes på

<https://produksjon.webdbs.no/Kontrakter/Kontrakter.htm>

6.2 Tjenestekontrakter

Siste versjon av kontrakts-dokumentene er tilgjengelige for alle på

<https://produksjon.webdbs.no/Kontrakter/Kontrakter.htm>

Forrige versjon av dokumentene er tilgjengelige på samme sted, men markert som utgått.